SBMS DATABASE SCHEMA AND DATA DICTIONARY

Prepared: 2026-11-02 **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Sources read:** `backend/src/db/schema.sql` (baseline), all 14 forward migrations in `backend/src/db/migrations/`, `backend/scripts/migrate.js`, and the table list measured during the backup rehearsal (`docs/database-recovery-rehearsal-2026-10-31.md` §2). **Machine:** PostgreSQL 17.11, database `sbms`, extension `pgcrypto` enabled for `gen_random_uuid()`.

---

1. Purpose and method

This file is the reference data dictionary for the report. It states, for every table and column that exists at this revision: the type, nullability, default, the constraint that governs it and what it means in the bursary workflow. It also records the state machines the schema enforces and, separately, the integrity rules the schema does **not** enforce, so a reader cannot mistake an application convention for a database guarantee.

Column definitions were read from `schema.sql` and from `ALTER TABLE ... ADD COLUMN` statements in the migrations. A column added by a migration is attributed to that migration rather than to the baseline file, because a database built from `schema.sql` alone will not contain it.

2. Schema organisation and migration policy

Property	Value
Baseline file	`backend/src/db/schema.sql` — creates `users`, `students`, `duplicate_flags`, `duplicate_review_events`, `disbursements`, `audit_logs`, `reports`, `beneficiaries` and applies idempotent `ALTER`s
Migration files	14 forward migrations, dated `20261019` to `20261031`, plus one rollback script `20261022_ml_duplicate_provenance.down.sql`
Application mechanism	`backend/scripts/migrate.js` applies unapplied files in filename order and records each in `sbms_schema_migrations`
Automatic application	**None.** The API never applies a migration at start-up; an operator applies them after backup and a verified restore
Fail-closed guard	`GET /api/ready` reports not ready when a required table is missing, including `bursary_certificates`
Migration count applied on the test database	12 on 2026-10-31 (before `20261023_retain_primary_accounts.sql` and `20261031_bursary_certificates.sql`); 14 after those were applied for the certificate work

Forward migration inventory

File	What it adds or changes
`20261019_admin_field_officer_roles.sql`	Constrains `users.role` to `admin` / `field_officer`
`20261019_duplicate_review_workflow.sql`	Creates `duplicate_review_events`; adds review columns to `duplicate_flags`
`20261019_enforce_supported_schools.sql`	Adds `students_school_supported_check`
`20261019_field_officer_registration.sql`	Supports officer-created accounts
`20261019_limit_synthetic_schools.sql`	Restricts synthetic seeds to the five schools
`20261019_remove_duplicate_case_assignment.sql`	Drops case assignment to a person; review is unassigned
`20261019_standardize_student_records.sql`	Normalises student identifier columns

File	What it adds or changes
`20261019_synthetic_student_marker.sql`	Adds `students.is_synthetic`
`20261022_ml_duplicate_provenance.sql`	Adds `model_version`, `review_threshold`, `comparison_features` to `duplicate_flags`
`20261023_bursary_award_reconciliation.sql`	Creates `bursary_awards`, `bursary_award_verification_events`, `disbursement_reconciliation_events`; adds 14 reconciliation columns to `disbursements` and six CHECK constraints
`20261023_retain_primary_accounts.sql`	Keeps primary (non-duplicate) accounts
`20261024_add_user_activation_flag.sql`	Adds `users.is_active`
`20261025_legacy_beneficiaries_baseline.sql`	Establishes the legacy `beneficiaries` baseline
`20261031_bursary_certificates.sql`	Creates `bursary_certificates` with snapshot and revocation columns

3. Table inventory

#	Table	Purpose	Primary key	Rows on the test database, 2026-10-31
1	`users`	Staff accounts: administrators and field officers	`id` UUID	4
2	`students`	Beneficiary register	`id` UUID	0
3	`duplicate_flags`	One review case per student pair, with model provenance	`id` UUID; unique unordered pair	0
4	`duplicate_review_events`	Append-only adjudication history per case	`id` UUID	0
5	`bursary_awards`	Award application, verification and decision	`id` UUID	0
6	`bursary_award_verification_events`	Append-only verification history per award	`id` UUID	0
7	`disbursements`	Payments issued against an award, with clearance and receipt fields	`id` UUID	0
8	`disbursement_reconciliation_events`	Append-only payment and receipt events	`id` UUID	0
9	`bursary_certificates`	Issued certificate snapshot with token and revocation	`id` UUID; `award_id` UNIQUE	0
10	`audit_logs`	Action trail across all controllers	`id` UUID	1
11	`reports`	Export metadata (type, range, file path)	`id` UUID	0
12	`beneficiaries`	Legacy module, serial key, no foreign keys	`id` SERIAL	0
13	`sbms_schema_migrations`	Applied-migration ledger	filename	12 then 14

The database therefore holds **13 tables** at this revision. The rehearsal measured 12 tables and 118 columns before `bursary_certificates` was added; the certificate table adds 15 more columns, giving 133 by arithmetic — a derived figure, not a re-measured one.

4. Data dictionary

4.1 `users` — staff accounts

Column	Type	Null	Default	Constraint	Meaning
`id`	UUID	no	`gen_random_uuid()`	PK	Account identifier; the subject of the JWT and the actor on every audit row

Column	Type	Null	Default	Constraint	Meaning
`full_name`	VARCHAR(255)	no	—	—	Name shown on staff screens
`email`	VARCHAR(255)	no	—	UNIQUE	Sign-in identity; lower-cased by the API before storage
`password_hash`	TEXT	no	—	—	bcrypt hash, cost 12; the plaintext is never stored or logged
`role`	VARCHAR(50)	no	`field_officer`	`users_role_supported_check` IN (`admin`, `field_officer`)	Authorisation role; re-read on every request, never trusted from the token
`is_active`	BOOLEAN	no	TRUE	—	Deactivation flag (`20261024_add_user_activation_flag.sql`); an inactive user cannot authenticate
`created_at`	TIMESTAMP	yes	`NOW()`	—	Registration time

4.2 `students` — beneficiary register

Column	Type	Null	Default	Constraint	Meaning
`id`	UUID	no	`gen_random_uuid()`	PK	Student identifier used by every downstream table
`full_name`	VARCHAR(255)	no	—	—	Name as recorded at the counter; whitespace collapsed to single spaces
`date_of_birth`	DATE	yes	—	—	ISO date; feeds the `dob_match` feature
`school_code`	VARCHAR(100)	yes	—	`students_school_supported_check` IN the five approved schools	School membership; feeds `school_match` and gates award submission
`guardian_id`	VARCHAR(100)	yes	—	—	Guardian identifier; feeds `guardian_match`
`upi`	VARCHAR(100)	yes	—	not unique (uniqueness deliberately removed)	Unique Personal Identifier; feeds `upi_match` and the critical identifier rule
`national_id_number`	VARCHAR(100)	yes	—	—	National identifier held for disclosure control and review evidence; not a model feature
`primary_phone`	VARCHAR(50)	yes	—	API constrains to `+254[17]` plus eight digits	Contact number
`qr_token`	VARCHAR(100)	yes	—	`students_qr_token_unique` partial unique index	64 lowercase hex characters generated at registration; never returned by the API
`is_synthetic`	BOOLEAN	no	FALSE	—	Marks seeded demonstration rows so they can be separated from real registrations
`birth_certificate_number`	VARCHAR(100)	yes	—	—	Birth certificate entry number; feeds `birth_certificate_match`
`eligibility_score`	INTEGER	yes	—	—	Last computed score (0-100)
`eligibility_category`	VARCHAR(50)	yes	—	—	`highly_eligible`, `eligible`, `partially_eligible`, `not_eligible`
`eligibility_assessed_at`	TIMESTAMP	yes	—	—	When the score was computed
`created_at`	TIMESTAMP	yes	`NOW()`	—	Registration time; used as the candidate ordering key during screening

Note: the baseline file renames `birth_certificate_entry_number` to `birth_certificate_number` if the old column exists, and drops the historical `students_upi_key` uniqueness constraint. Both actions are idempotent and are recorded in `schema.sql:47-72`.

4.3 `duplicate_flags` — review cases

Column	Type	Null	Default	Constraint	Meaning
`id`	UUID	no	`gen_random_uuid()`	PK	Case identifier
`student_id`	UUID	no	—	FK `students(id)` ON DELETE CASCADE	The newly registered student
`matched_student_id`	UUID	no	—	FK `students(id)` ON DELETE CASCADE	The existing student it was compared with
`probability`	NUMERIC(5,4)	no	—	—	**Legacy name**: holds the uncalibrated positive-class model score, as documented in the column comment
`status`	VARCHAR(50)	no	`pending`	—	`pending`, `confirmed_duplicate`, `not_duplicate`, `needs_verification`
`model_version`	TEXT	yes	—	—	Artefact identity, e.g. `xgboost:<sha256>`; proves which model produced the score
`review_threshold`	NUMERIC(5,4)	yes	—	—	Cutoff in force at scoring time
`comparison_features`	JSONB	yes	—	—	The six feature values used, retained as evidence
`resolved_by`	UUID	yes	—	FK `users(id)`	Administrator who decided the case
`resolved_at`	TIMESTAMP	yes	—	—	Decision time
`updated_at`	TIMESTAMP	no	`NOW()`	—	Last modification
`created_at`	TIMESTAMP	yes	`NOW()`	—	Case creation

A unique index on `LEAST(student_id, matched_student_id), GREATEST(student_id, matched_student_id)` (`duplicate_flags_unordered_pair_unique`) makes a pair unique regardless of direction, so the same two students cannot accumulate duplicate cases in both orientations.

4.4 `duplicate_review_events` — adjudication history

Column	Type	Null	Default	Constraint	Meaning
`id`	UUID	no	`gen_random_uuid()`	PK	Event identifier
`duplicate_flag_id`	UUID	no	—	FK `duplicate_flags(id)` ON DELETE CASCADE	Case the event belongs to
`reviewer_id`	UUID	no	—	FK `users(id)`	Acting administrator
`decision`	VARCHAR(30)	no	—	CHECK IN (`confirmed_duplicate`, `not_duplicate`, `needs_verification`)	Decision recorded
`reason`	TEXT	no	—	CHECK `length(btrim(reason)) > 0` at database level; API enforces >= 5 characters	Justification
`notes`	TEXT	yes	—	—	Free-text detail
`created_at`	TIMESTAMP	no	`NOW()`	—	When the decision was appended

Index `duplicate_review_events_flag_created_idx` on `(duplicate_flag_id, created_at DESC)` supports the per-case history view.

4.5 `bursary_awards` — application, verification and decision

Column	Type	Null	Default	Constraint	Meaning
`id`	UUID	no	`gen_random_uuid()`	PK	Award identifier

Column	Type	Null	Default	Constraint	Meaning
`student_id`	UUID	no	—	FK `students(id)` ON DELETE RESTRICT	Beneficiary the award belongs to
`school_code`	VARCHAR(100)	no	—	—	Copied from the student at submission so the award records the school it was granted under
`requested_amount`	NUMERIC(12,2)	no	—	CHECK `> 0`	Amount applied for
`approved_amount`	NUMERIC(12,2)	yes	—	CHECK NULL or `> 0`	Amount approved; also the ceiling for total disbursement
`status`	VARCHAR(20)	no	`submitted`	CHECK IN (`submitted`, `approved`, `rejected`)	Decision state
`verification_status`	VARCHAR(30)	no	`not_checked`	CHECK IN (`not_checked`, `verified`, `not_found`, `needs_investigation`)	Beneficiary verification state
`verification_notes`	TEXT	yes	—	—	Verification narrative
`verification_evidence_ref`	VARCHAR(500)	yes	—	Required when `verification_status = 'verified'`	Pointer to the evidence seen; explicitly **not** an upload
`verified_by`	UUID	yes	—	FK `users(id)` ON DELETE RESTRICT	Officer who verified
`verified_at`	TIMESTAMPTZ	yes	—	—	Verification time
`decision_reason`	TEXT	yes	—	CHECK >= 5 trimmed characters when decided	Why approved or rejected
`decided_by`	UUID	yes	—	FK `users(id)` ON DELETE RESTRICT	Administrator who decided
`decided_at`	TIMESTAMPTZ	yes	—	—	Decision time
`submitted_by`	UUID	yes	—	FK `users(id)` ON DELETE RESTRICT	Officer who submitted
`created_at`	TIMESTAMPTZ	no	`NOW()`	—	Submission time
`updated_at`	TIMESTAMPTZ	no	`NOW()`	—	Last modification

Table-level CHECK constraints (`bursary_awards_*`): a submitted row must have no decision fields; an approved row must have an amount, a reason, a decider and a time; a rejected row must have no amount but must have a reason, a decider and a time; `status <> 'approved' OR verification_status = 'verified'` — approval without verification is impossible at database level, not only in the controller; and a verified row must carry verifier, time and a non-empty evidence reference.

4.6 `bursary_award_verification_events` — verification history

Column	Type	Null	Default	Constraint	Meaning
`id`	UUID	no	`gen_random_uuid()`	PK	Event identifier
`award_id`	UUID	no	—	FK `bursary_awards(id)` ON DELETE RESTRICT	Award verified
`outcome`	VARCHAR(30)	no	—	CHECK IN (`verified`, `not_found`, `needs_investigation`)	Outcome of the check
`notes`	TEXT	no	—	CHECK >= 5 trimmed characters	What was checked
`evidence_ref`	VARCHAR(500)	yes	—	Required when `outcome = 'verified'`	Evidence pointer
`actor_id`	UUID	no	—	FK `users(id)` ON DELETE RESTRICT	Officer
`created_at`	TIMESTAMPTZ	no	`NOW()`	—	When recorded

4.7 `disbursements` — payments

Baseline columns:

Column	Type	Null	Default	Meaning
`id`	UUID	no	`gen_random_uuid()`	Payment identifier
`student_id`	UUID	no	—	FK `students(id)` ON DELETE CASCADE; payer-facing student
`amount`	NUMERIC(12,2)	no	—	Amount issued
`disbursement_date`	DATE	no	`CURRENT_DATE`	Issue date; basis of the overdue-acknowledgement calculation
`status`	VARCHAR(50)	no	`completed`	Legacy status, kept in step with `payment_status` by the update statement
`created_by`	UUID	yes	—	FK `users(id)`
`created_at`	TIMESTAMP	yes	`NOW()`	Record creation

Reconciliation columns added by `20261023_bursary_award_reconciliation.sql`:

Column	Type	Constraint	Meaning
`award_id`	UUID	FK `bursary_awards(id)` ON DELETE RESTRICT	Award the payment settles; NULL for a legacy payment
`payment_method`	VARCHAR(80)	—	How it was paid (API requires 2-80 characters)
`payment_reference`	VARCHAR(150)	—	Bank or mobile-money reference
`payment_status`	VARCHAR(20)	CHECK `issued` | `cleared` | `failed` | `cancelled`	Settlement state
`cleared_by`	UUID	FK `users(id)`	Who confirmed settlement
`cleared_at`	TIMESTAMPTZ	—	When settlement was confirmed
`clearance_evidence_ref`	VARCHAR(500)	—	Evidence for clearance
`receipt_status`	VARCHAR(30)	CHECK `awaiting_acknowledgement` | `acknowledged` | `discrepancy`	Receipt state
`acknowledged_amount`	NUMERIC(12,2)	CHECK `>= 0`	Amount the officer recorded as received
`acknowledged_by`	UUID	FK `users(id)`	Who recorded the receipt
`acknowledged_at`	TIMESTAMPTZ	—	When it was recorded
`acknowledgement_method`	VARCHAR(30)	CHECK must equal `officer_assisted` when acknowledged	Records that the acknowledgement is officer-assisted, not school-confirmed
`acknowledgement_evidence_ref`	VARCHAR(500)	—	Evidence for the acknowledgement
`acknowledgement_notes`	TEXT	—	Free text

Two table CHECK constraints make the states self-consistent: `disbursements_clearance_fields_check` requires `cleared_by`, `cleared_at` and a non-empty `clearance_evidence_ref` if and only if `payment_status = 'cleared'`; `disbursements_acknowledgement_fields_check` requires amount, actor, time, method `officer_assisted` and evidence if and only if `receipt_status` is `acknowledged` or `discrepancy`. A half-recorded clearance or receipt is therefore impossible.

4.8 `disbursement_reconciliation_events` — payment and receipt trail

Column	Type	Null	Constraint	Meaning
`id`	UUID	no	PK	Event identifier

Column	Type	Null	Constraint	Meaning
`disbursement_id`	UUID	no	FK `disbursements(id)` ON DELETE RESTRICT	Payment the event belongs to
`event_type`	VARCHAR(30)	no	CHECK IN (`payment_issued`, `payment_cleared`, `payment_failed`, `payment_cancelled`, `payment_linked_to_award`, `receipt_acknowledged`)	What happened
`payment_status`	VARCHAR(20)	yes	CHECK as above	Status after the event
`receipt_status`	VARCHAR(30)	yes	CHECK as above	Receipt state after the event
`amount`	NUMERIC(12,2)	yes	CHECK `>= 0`	Amount associated with the event
`evidence_ref`	VARCHAR(500)	yes	—	Evidence pointer
`notes`	TEXT	yes	—	Reason or note
`actor_id`	UUID	no	FK `users(id)` ON DELETE RESTRICT	Who acted
`created_at`	TIMESTAMPTZ	no	`NOW()`	When

4.9 `bursary_certificates` — issued certificate snapshot

Column	Type	Null	Constraint	Meaning
`id`	UUID	no	PK	Certificate row identifier
`award_id`	UUID	no	FK `bursary_awards(id)` ON DELETE RESTRICT, **UNIQUE**	One certificate per award; a second issue is refused with 409
`certificate_reference`	VARCHAR(40)	no	UNIQUE	`SBMS-CERT-<year>-<8 uppercase hex>`; the hex part is random, not a sequence
`verification_token`	VARCHAR(100)	no	UNIQUE, CHECK `^[a-f0-9]{64}$`	Public token; lives in the URL fragment and is never logged
`award_period`	VARCHAR(20)	no	CHECK `^[0-9]{4}(/[0-9]{4})?$`	`2026` or `2025/2026`
`student_name`	VARCHAR(255)	no	—	Snapshot of the printed name
`student_upi`	VARCHAR(100)	yes	—	Snapshot of the UPI
`school_code`	VARCHAR(100)	no	—	Snapshot of the school
`approved_amount`	NUMERIC(12,2)	no	CHECK `> 0`	Snapshot of the amount; verification compares it with the live award
`award_approved_at`	TIMESTAMPTZ	no	—	Snapshot of the approval time
`status`	VARCHAR(20)	no	CHECK IN (`active`, `revoked`)	Certificate state
`issued_by`	UUID	no	FK `users(id)` ON DELETE RESTRICT	Issuing administrator
`issued_at`	TIMESTAMPTZ	no	—	Issue time
`revoked_by`	UUID	yes	FK `users(id)` ON DELETE RESTRICT	Revoking administrator
`revoked_at`	TIMESTAMPTZ	yes	—	Revocation time
`revocation_reason`	TEXT	yes	CHECK >= 5 trimmed characters when revoked	Why revoked

`bursary_certificates_revocation_check` enforces that an active row has no revocation fields and a revoked row has all three. The table exists so that a later edit to the award cannot silently rewrite what a printed document says; the verification endpoint compares the snapshot with the live award and answers `not_current` on a mismatch.

4.10 `audit_logs`, `reports`, `beneficiaries`, `sbms_schema_migrations`

Table	Column	Type	Null	Constraint	Meaning
`audit_logs`	`id`	UUID	no	PK	Event identifier
	`user_id`	UUID	yes	FK `users(id)`	Actor; NULL only for a system action
	`action`	VARCHAR(100)	no	—	Verb, e.g. `CREATE`, `AWARD_APPROVED`, `PAYMENT_CLEARED`, `AWARD_CERTIFICATE_ISSUED`
	`table_name`	VARCHAR(100)	yes	—	Affected table
	`record_id`	UUID	yes	—	Affected record
	`details`	JSONB	yes	—	Context: model version, thresholds, counters, reasons
	`created_at`	TIMESTAMP	yes	`NOW()`	When
`reports`	`id`	UUID	no	PK	Export identifier
	`generated_by`	UUID	yes	FK `users(id)`	Administrator who exported
	`report_type`	VARCHAR(100)	no	—	Export kind
	`start_date`, `end_date`	DATE	yes	—	Range requested
	`file_path`	TEXT	yes	—	Path recorded at generation; the export is streamed, not persisted
	`created_at`	TIMESTAMP	yes	`NOW()`	When
`beneficiaries`	`id`	SERIAL	no	PK	Legacy integer key
	`name`	VARCHAR(255)	no	—	Legacy name
	`school`	VARCHAR(255)	yes	—	Legacy school text; not a foreign key to `students.school_code`
	`grade_level`	VARCHAR(50)	yes	—	Legacy field with no counterpart in `students`
	`scholarship_amount`	NUMERIC	yes	default 0	Legacy amount; meaning unresolved and deliberately not mapped to any award or payment
	`created_at`	TIMESTAMP	yes	`NOW()`	When
`sbms_schema_migrations`	`filename`	text	no	PK	Migration applied
	`applied_at`	timestamp	—	—	When `migrate.js` applied it

5. Relationships and referential actions

Parent	Child	Column	On delete	Consequence
`students`	`duplicate_flags`	`student_id`	CASCADE	Deleting a student removes its cases
`students`	`duplicate_flags`	`matched_student_id`	CASCADE	Cases disappear when either party is deleted
`duplicate_flags`	`duplicate_review_events`	`duplicate_flag_id`	CASCADE	Review history dies with the case
`users`	`duplicate_flags`	`resolved_by`	no action	A deciding administrator cannot be deleted while cases reference them
`users`	`duplicate_review_events`	`reviewer_id`	no action	Same
`students`	`bursary_awards`	`student_id`	RESTRICT	A student with an award cannot be deleted
`bursary_awards`	`bursary_award_verification_events`	`award_id`	RESTRICT	Verification history blocks deletion
`bursary_awards`	`disbursements`	`award_id`	RESTRICT	A paid award cannot be deleted

Parent	Child	Column	On delete	Consequence
`bursary_awards`	`bursary_certificates`	`award_id`	RESTRICT, UNIQUE	A certified award cannot be deleted and cannot be certified twice
`disbursements`	`disbursement_reconciliation_events`	`disbursement_id`	RESTRICT	Payment history blocks deletion
`students`	`disbursements`	`student_id`	CASCADE	Legacy behaviour retained; the RESTRICT award link still applies when `award_id` is set

The asymmetry is deliberate: review data cascades because it is derived from the register, while money, certification and verification data restrict deletion because they are accountability records. Deleting a student who holds an award therefore fails at the database, not only in the application.

6. Indexes and their purpose

Index	Table (columns)	Purpose
`students_qr_token_unique`	`students(qr_token)` WHERE NOT NULL	Public verification lookup by token; partial so rows without a token do not collide
`duplicate_flags_unordered_pair_unique`	`duplicate_flags(LEAST(...), GREATEST(...))`	One case per unordered pair
`duplicate_review_events_flag_created_idx`	`duplicate_review_events(duplicate_flag_id, created_at DESC)`	Case history, newest first
`bursary_awards_status_created_idx`	`bursary_awards(status, created_at DESC)`	Queue of submitted awards and their age
`bursary_awards_student_created_idx`	`bursary_awards(student_id, created_at DESC)`	A student's award history
`bursary_award_verification_events_award_idx`	`(award_id, created_at DESC)`	Verification history per award
`disbursements_award_status_idx`	`disbursements(award_id, payment_status, receipt_status)`	Reconciliation views and exception queries
`disbursement_reconciliation_events_disbursement_idx`	`(disbursement_id, created_at DESC)`	Payment event trail
`bursary_certificates_status_issued_idx`	`(status, issued_at DESC)`	Active versus revoked certificate listing

No index exists on `students.full_name`, `students.school_code` or `students.date_of_birth`. That is consistent with FR-34: screening reads the whole table, so no index would be used. It is also why a blocking strategy (D4 in the ML decision sheet) would require new indexes.

7. State machines enforced by the schema

Entity	Transition	Enforced by	Cannot happen
Duplicate case	`pending` to `confirmed_duplicate` / `not_duplicate` / `needs_verification`, then terminal	Controller refuses a second decision; `duplicate_review_events` appends history	Re-deciding a terminal case returns 404
Award	`submitted` to `approved` or `rejected`	`bursary_awards_decision_fields_check`; approval additionally requires `verification_status = 'verified'`	Approval of an unverified award; a decision without a reason; an amount on a rejected award
Verification	`not_checked` to `verified` / `not_found` / `needs_investigation`	`bursary_awards_verification_fields_check`; the controller closes verification after a decision	A `verified` outcome without evidence, actor and time

Entity	Transition	Enforced by	Cannot happen
Payment	`issued` to `cleared` / `failed` / `cancelled`, with no further change	`bursaryRules.canTransitionPayment`; `disbursements_clearance_fields_check`	Clearing twice; failing a payment that already has a receipt status
Receipt	NULL to `awaiting_acknowledgement` to `acknowledged` or `discrepancy`	`bursaryRules.receiptStatus` compares cents; `disbursements_acknowledgement_fields_check`	An acknowledged receipt without amount, actor, time and evidence; any method other than `officer_assisted`
Certificate	`active` to `revoked`	`bursary_certificates_revocation_check`	Revoking twice; reissuing for the same award (`award_id` UNIQUE)

Amount comparison is performed in integer cents (`bursaryRules.cents`), so a floating-point difference cannot turn an exact receipt into a discrepancy, or the reverse.

8. Legacy and dead schema objects

Object	State	Handling
`duplicate_flags.probability`	Legacy name for an uncalibrated model score	Documented by a column comment; the API exposes the value as `model_score`
`disbursements.status`	Legacy duplicate of `payment_status`	Kept in step by the update statement; not the authoritative field
`beneficiaries` table	Legacy module, serial key, no foreign keys, `scholarship_amount` with unresolved meaning	Retained because `/api/beneficiaries` and the readiness contract require it; explicitly not mapped to awards or payments
Historical `students_upi_key` uniqueness	Dropped in `schema.sql:47-49`	Duplicate UPI values are now permitted, because duplicate detection must be able to observe them as a signal
`birth_certificate_entry_number`	Renamed to `birth_certificate_number` where it exists	Idempotent rename block in `schema.sql:59-72`
Case-assignment column	Removed by `20261019_remove_duplicate_case_assignment.sql`	Review is unassigned by design; any administrator may take a case

9. Data provenance and synthetic data

Marker or record	Meaning
`students.is_synthetic = TRUE`	Row created by a seed script, not by a registration; separates demonstration data from registrations
`docs/repository-cleanup-2026-10-31.md`	Records the cleanup that limited synthetic schools to the five approved names
Deployment demonstration data	Contains records created during live checks (`docs/azure-deployment-log-2026-10-31.md` §7-§9), including two `E2E Probe` students and one case the log instructs the operator to delete
No real beneficiary data	The repository and the test database contain no real student data; every figure in this documentation set must be produced from synthetic rows (G-05)

10. What the schema does not enforce

Gap	Consequence	Recorded as
No append-only protection on `audit_logs`, `duplicate_review_events`, `bursary_award_verification_events`, `disbursement_reconciliation_events`	Immutability is by convention; a database user with UPDATE or DELETE rights can rewrite history	NFR-21; `docs/business-workflow-verification-2026-10-30.md` §5
No retention or archival policy	Rows are never aged out; no partition or purge strategy exists	G-19

Gap	Consequence	Recorded as
No uniqueness on `students.upi`	By design, but a UPI is therefore not a key and cannot be used to join reliably	§8
No row-level security	Access control is entirely in the application; any client holding database credentials bypasses it	`SBMS_SECURITY_AND_ACCESS_CONTROL_CATALOGUE.md` §8
No constraint tying `disbursements.student_id` to `bursary_awards.student_id`	The controller checks the pairing; the database does not	Future work
`beneficiaries` has no relationship to any other table	Two parallel registries exist	§8
No trigger maintaining `updated_at`	Every update path must set it; a direct `UPDATE` leaves it stale	Inspection finding

11. Reproduction

```
# schema only (no data), against a disposable database
psql -h localhost -U <owner> -d <disposable> -f backend/src/db/schema.sql
node backend/scripts/migrate.js          # applies unapplied files in filename order
psql -h localhost -U <owner> -d <disposable> -c "\dt"        # 13 tables expected
psql -h localhost -U <owner> -d <disposable> -c "SELECT * FROM sbms_schema_migrations ORDER BY filename"
```

Applying migrations to the development or deployment database is an operator action with backup and a verified restore first; the application never does it automatically.